

Integer and Logical Operations: RPEG

Designed by:

Aeyva Rebelo

main.rs Module

Input Handling

- Program Usage: rpeg [flags] [filename]
 - [flags]
 - -d: Decompress procedure.
 - -c: Compression procedure.
 - -t: Round-trip procedure.
 - This flag is available, but is mainly used for debugging purposes.
 - [filename]
 - The name of the file to be decompressed, if using a PPM file to be compressed with the -c flag, or a compressed image file to be decompressed with the -d flag.
 - If no filename is provided, the program will read from standard input.
 - The round trip flag will not work with standard input.
- Upon running the program with the proper usage, the main program will call the relevant function from the '**Codec.rs**' module, with the filename passed into the command line interface.

Input Files

- Compression: Standard PPM file format
 - Magic Number/Header: P6
 - Dimensions: '[Width] [Height]'
 - Denominator: 255
 - This will be the only supported denominator for this implementation.
 - Data: [Width] * [Height] sized 2-D vector of RGB values in the range from 0-255 (the standard denominator).
- Decompression: Compressed file format as specified in assignment doc.
 - Header: "Compressed image format 2"
 - Dimensions: '[Width] [Height]'
 - Data: [Width]*[Height] amount of binary codewords as packed in the compression module.

Codec.rs Module

Compress - (compress)

- Arguments: Filename
- Returns: N/a
- Result: Compresses a PPM image and writes the list of bit codewords to the standard output along with the specified header, width, and height of the original image.
- Unit Tests:
 - Include sample PPM images to test the full functionality of every module utilized in the compression algorithm.
 - Verification of input handling (both command line and std::in), formatting, and string handling

Decompress - (decompress)

- Arguments: Filename
- Returns: N/a
- Result: Decompresses an image in compressed image format 2 back into PPM format, printing the result to standard output.
- Unit Tests:
 - Include sample PPM images which will be compressed to test the fully reversed functions developed in tandem with the compression algorithms' functions.
 - Verification of input handling (both command line and std::in), formatting, and string handling

Round Trip - (round_trip)

- Arguments: Filename
- Returns: N/a
- Result: Compresses and then decompresses a PPM image according to the compression and decompression procedure to "output.ppm" for bug testing.
- Unit Tests:
 - Tests for this module will be made up of all the tests that each sub-task of the compression and decompression algorithm require.

Image.rs Module

Image Trimming - (image_trim)

- Arguments: &mut RgbImage
- Returns: N/a
- Result: Checks to see if the dimensions of the referenced PPM image's data are even or not, and trims it to be of even dimensions if not.
- Unit Tests:
 - Check to see that images of both uneven and even dimensions are properly sized.

Compressed Image Reading - (read_image_data)

- Arguments: Filename
- Returns: WordList struct with populated list of words, width, and height.
- Result: Reads the image data of a compressed image file or through stdin, and creates a WordList for which to be processed for decompression.
- Invariant(s):
 - The image dimensions do not have to be checked, as they must be even since any properly formatted compressed image file will be of even dimensions.
- Unit Tests:
 - Check that the result of passing in a compressed image's file is the same as directly passing the values of the data from the roundtrip function.

Rgb_cvc.rs Module

Rgb Struct - (Inherited from cargo csc411_image)

- Field(s):
 - red: u16
 - green: u16
 - blue: u16

RGB to RGBF64 - (rgb_to_rgbf64)

- Arguments: Vec<Rgb>
- Returns: Vec<(f64, f64, f64)>
- Result: Maps a vector of Rgb structs' pixel values into floating point values, and collects them in a tuple.
- Unit Tests:
 - Ensure that the values are properly mapped and are of the correct value.

RGBF64 to Component Video Code - (rgbf64_to_component_video)

- Arguments: Vec<(f64, f64, f64)>
 - Vector of R, G, B values of type f64 wrapped in a vector of tuples.
- Returns: Vec<(f64, f64, f64)>
 - Vector of Y, Pb, Pr values of type f64 wrapped in a vector of tuples.
- Result: Iterates over a vector of RGB structs of type f64 to calculate and map them into a tuple of Y, Pb, and Pr values, which are equivalent to the RGB values in component video code.
- Unit Tests:
 - Ensure that the values are being correctly calculated against known values.

Component Video Code to RGBF64 - (component_video_to_rgbf64)

- Arguments: Vec<(f64, f64, f64)>
 - Vector of Y, Pb, Pr values of type f64.
- Returns: Vec<(f64, f64, f64)>
 - Vector of R, G, B values of type f64.
- Result: Iterates and maps the component video code values into an equivalent vector of RGB format.
- Unit Tests:
 - Ensure that round-trip capabilities work between RgbF64 -> CvC -> RgbF64 format.

RGBF64 to RGB - (rgbf64_to_rgb)

- Arguments: Vec<(f64, f64, f64)>
 - Vector of R, G, B values of type f64 wrapped in a vector of tuples.
- Returns: Vec<Rgb>
 - Vector of Rgb structs.
- Result: Converts the vector of RgbF64 back into a vector of Rgb structs, rounding the values back to their closest integer.
- Unit Tests:
 - Ensure that there is an approximate equivalence between the original floating point values and the integer values after conversion.

Cvc_blocks.rs Module

Component Video Code Struct - (CvcBlock)

- Field(s):
 - width: usize
 - height: usize
 - pixels: Vec<((f64, f64, f64),(f64, f64, f64),(f64, f64, f64),(f64, f64, f64))>
 - Vector of four tuples of Y, Pb, Pr values, meant to hold the sub-divided image of cvc tuples in blocks of 2x2.

Component Video to Blocks - (cvc_to_blocks)

- Arguments: Vec<(f64, f64, f64)>, height, width
 - Vector of tuples of Y, Pb, Pr values passed from rgbf64_to_component_video(...)
- Returns: CvcBlock
 - Struct that holds pixel values in 2x2 blocks of 'cvc' tuples, width, and height of original image.
- Result: Subdivides the original image's data into 2x2 vectors which hold the tuples of its component video code values.
- Unit Tests:
 - Ensure that the items inside of each tuple of vectors of f64 values match the same values of the original vector of tuples.

Blocks to Component Video - (blocks_to_cvc)

- Arguments: CvcBlock
 - List of 2x2 tuples of Y, Pb, Pr pixels.
- Returns: Vec<(f64, f64, f64)>
 - List of tuples of Y, Pb, Pr pixels.
- Result: Rebuilds the list of component video code blocks by deconstructing the CvcBlock struct into a 1-dimensional vector.
- Unit Tests:
 - Ensure that round-trip functionality results in the equivalent arrangement of data from cvc -> cvc blocks -> cvc.

Blocks_bwc.rs Module

List of Bit Codewords Struct - (WordList)

- Field(s):
 - words: Vec<u32>
 - List of 32-bit compressed codewords compressed with the following words according to specification:

Value	Type	Width	LSB
a	Unsigned scaled integer	9 bits	23
b	Signed scaled integer	5 bits	18
c	Signed scaled integer	5 bits	13
d	Signed scaled integer	5 bits	8
$\text{index}(\overline{P_B})$	Unsigned index	4 bits	4
$\text{index}(\overline{P_R})$	Unsigned index	4 bits	0

- width: usize
- height: usize

Blocks to Bit Word Code - (block_to_bwc)

- Arguments: CvcBlock
- Returns: WordList
- Result: Using the list of pixels of the image divided into 2x2 groups from the CvcBlock object, calls the pack_word(...) function and returns the list of encoded bit words that make up the entirety of the image.
- Unit Tests:
 - Ensure that round-trip testing allows the individual pixel values to be encoded and decoded into the exact same values, with respect to rounding and clamping.

Bit Word Code to Block - (bwc_to_block)

- Arguments: WordList
- Returns: CvcBlock
- Result: For each word in the WordList, unpacks and decodes each word back into 2x2 blocks from the values stored in the bit word code.
- Unit Tests:
 - Ensure that sample values are decoded into their proper values, and applied correctly to the 2x2 CvcBlock pixels.

Unpack Word - (unpack_word)

- Arguments: u64
 - The “32-bit” bit encoded word that is simply a 64-bit value for the purposes of the expected type as a part of bitpack.
- Returns: u64, i64, i64, i64, f64, f64
 - a, b, c, d, PbAvg, PrAvg
- Result: Unpacks and returns each of the fields by unshifting the bit word and utilizing chroma_of_index from the csc411_arith crate as necessary for the Pb and Pr values.
- Unit Tests:
 - Ensure that a sample word can be unpacked properly into its fields.

Pack Word - (pack_word)

- Arguments: u64, i64, i64, i64, f64, f64
- Returns: u64
- Result: Packs and returns the value of the encoded bit word formed from all of the components passed through as arguments.
- Unit Tests:
 - Ensure that a sample set of values can be packed properly into its fields, and that a round trip with unpack_word(...) provides the same values that were packed in.

Bitpack.rs Module

Disclaimer: The bitpack module that was worked on during the original project development during the semester is the same bitpack module that was used for the completion of this assignment. As such, it was worked on alongside Alex Bergeron and Kai Maffucci, and acknowledgement goes to them for the usage of the module. Further, part of this documentation will be borrowed from the previous design document worked on between Alex and I.

Fits into 'Width' Signed Bits - (fitss)

- Arguments: i64, u64
 - Signed integer value, width of a bit field
- Returns: bool
- Result: Returns true or false based on whether the signed integer value will fit inside of the width of a bit field.
- Unit Tests:
 - Verify that positive and negative integers correctly fit or do not fit within bit widths.

Fits into 'Width' Unsigned Bits - (fitsu)

- Arguments: u64, u64
 - Unsigned integer value, width of a bit field
- Returns: bool
- Result: Returns true or false based on whether the unsigned integer value will fit inside of the width of a bit field.
- Unit Tests:
 - Test to ensure unsigned ints are correctly evaluated for fitting within specific bit widths.

Get from 'Width' Signed Bits - (gets)

- Arguments: u64, u64, u64
 - Unsigned bit encoded word, width of a bit field, least-significant bit
- Returns: Option<i64>
- Result: Returns the value of the signed integer from the specified fields of a word.
- Unit Tests:
 - Ensure that sample signed bits are correctly extracted from a test word.

Get from 'Width' Unsigned Bits - (getu)

- Arguments: u64, u64, u64
 - Unsigned bit encoded word, width of a bit field, least-significant bit of bit field
- Returns: Option<u64>
- Result: Returns the value of the unsigned integer from the specified fields of a word.
- Unit Tests:
 - Ensure that sample unsigned bits are correctly extracted from a test word.

Returns New Word with Unsigned Bits - (newu)

- Arguments: u64, u64, u64, u64
 - Unsigned bit encoded word, width of a bit field, least-significant bit of bit field, unsigned value to place into that bit field
- Returns: Option<u64>
 - Modified version of the unsigned word
- Result: Modifies the word to have the value of the unsigned value at the specified location.
- Unit Tests:
 - Ensure that inserting unsigned bits does not compromise the integrity of the encoded words' values after modification.

Returns New Word with Signed Bits - (news)

- Arguments: u64, u64, u64, i64
 - Unsigned bit encoded word, width of a bit field, least-significant bit of bit field, signed value to place into that bit field
- Returns: Option<u64>
 - Modified version of the unsigned word
- Result: Modifies the word to have the value of the signed value at the specified location.
- Unit Tests:
 - Ensure that inserting signed bits does not compromise the integrity of the encoded words' values after modification.